

BMSBorůvka: A Multi-Level Bucket Framework for Minimum Spanning Tree

Abstract

The minimum spanning tree (MST) problem traditionally relies on global edge sorting, leading to an $O(m \log n)$ time complexity barrier in algorithms such as Kruskal's Algorithm and Prim's Algorithm. In this paper, we propose BMSBorůvka, a bucket-based Borůvka-style algorithm that avoids full sorting through phased edge activation and multi-level graph compression. Our approach achieves $O(m \log^{\frac{2}{3}} n)$ time on sparse graphs while maintaining $O(m \log n)$ performance in the general case.

1 Introduction

A minimum spanning tree (MST) or minimum weight spanning tree is a subset of the edges of a connected, edge-weighted undirected graph that connects all the vertices together, without any cycles and with the minimum possible total edge weight. Classical algorithms such as Prim's and Kruskal's algorithms achieve optimal or near-optimal performance in general settings, but they rely heavily on comparison-based mechanisms—either through repeated priority queue operations in Prim's algorithm or through explicitly sorting all edges by weight in Kruskal's algorithm. As a result, their running time is dominated by comparison-based sorting costs, typically on the order of $O(m \log n)$, where n is the number of vertices in the graph.

This reliance reflects a broader sorting bottleneck that also appears in shortest path algorithms. It has long been known that when values can be classified in constant time, sorting can be performed in linear time, avoiding comparison-based ordering. Recent work [DMMS+25] on breaking the sorting barrier for directed single-source shortest paths exploits this idea by replacing comparison-heavy priority queue operations with classification-based techniques using multi-level buckets and graph sparsification. Motivated by this approach, we extend similar ideas to the Borůvka framework and demonstrate improved performance.

2 Related Work

The minimum spanning tree (MST) problem is one of the most fundamental problems in graph algorithms, with a long line of research focusing on improving its time complexity. Classical algorithms such as Kruskal [Kru56] and Prim [Pri57] algorithms achieve $O(m \log n)$ time by relying on comparison-based sorting or priority queue operations.

A major line of work has sought to reduce or bypass the sorting bottleneck inherent in these approaches. Chazelle [Cha00] gave a deterministic MST algorithm running in $O(m \alpha(m, n))$ time, where α is the inverse Ackermann function. His algorithm avoids explicit global sorting by using soft heaps, effectively allowing controlled key corruption to achieve near-linear performance.

Randomization has also played a crucial role in overcoming the sorting barrier. Karger, Klein, and Tarjan [KKT95] presented a randomized linear-time MST algorithm with expected running time $O(m)$, combining random sampling, filtering, and contraction techniques. This result demonstrates that, in the randomized setting, the sorting barrier can be circumvented entirely.

Pettie and Ramachandran [PR02] further advanced the theoretical understanding of MST by designing an optimal comparison-based algorithm. Their algorithm achieves a running time of $O(m \alpha(m, n))$ plus a term matching the decision-tree complexity of MST, thereby showing that MST can be solved optimally in the comparison model without requiring explicit sorting.

Despite these advances, most near-linear or optimal algorithms rely on sophisticated data structures, randomization, or implicit sorting mechanisms. In contrast, recent work on single-source shortest paths (SSSP) has explored breaking the sorting barrier through hierarchical bucketing and sparsification techniques [DMMS+25]. Inspired by these developments, our work investigates whether similar ideas can be applied to MST construction.

In particular, we propose a bucket-based multi-level sparsification framework that groups edges by weight ranges and progressively reduces the graph while preserving candidate MST edges. This approach avoids global edge sorting and instead relies on structured classification and local processing, drawing conceptual parallels to Borůvka-style algorithms. Our goal is to achieve a theoretically sound and practically efficient alternative that reshapes the role of sorting in MST algorithms.

3 Preliminaries

3.1 Graphs and the Minimum Spanning Tree Problem

An undirected weighted graph is a triple $G = (V, E, w)$ where V is a finite set of vertices, $E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is a set of edges, and $w : E \rightarrow \mathbb{Z}$ assigns an integer weight to each edge. We write $n = |V|$ and $m = |E|$.

A path between two vertices u and v is a sequence of distinct vertices $u = v_0, v_1, \dots, v_k = v$ such that $\{v_i, v_{i+1}\} \in E$ for each i . A graph is connected if a path exists between every pair of vertices; otherwise it is disconnected.

A spanning tree connects all vertices of a graph with no cycles; when edges carry weights, the spanning tree that minimises the sum of edge weights is called the minimum spanning tree (MST) [Hil24]. A spanning tree of a connected graph G is a subgraph $T \subseteq E$ that (i) connects all n vertices, (ii) contains no cycles, and (iii) has exactly $n - 1$ edges. A minimum spanning tree (MST) is a spanning tree whose total edge weight $\sum_{e \in T} w(e)$ is minimised over all spanning trees. If G is disconnected with k connected components, the analogous structure is a minimum spanning forest (MSF), which has $n - k$ edges.

Two classical properties are central to all MST algorithms:

Cut Property. Let $S \subset V$ be any non-empty proper subset of vertices. The cut defined by S consists of all edges with one endpoint in S and the other in $V \setminus S$. If e is the unique minimum-weight edge of this cut, then e belongs to every MST. More generally, any minimum-weight edge across any cut can be safely added to a partial MST without creating a suboptimal solution.

Cycle Property. If e is the unique maximum-weight edge on some cycle in G , then e does not belong to any MST.

These two properties together characterise exactly the edges that belong to the MST.

3.2 Borůvka's Algorithm

Borůvka's algorithm [Bor26] is the foundation on which BMSBoruvka is built. The key idea is to grow many components of the MST simultaneously rather than one edge at a time.

Starting with n isolated components (one per vertex), the algorithm repeats the following Borůvka round until only one component remains:

Find cheapest outgoing edge. For each current component, identify its minimum-weight edge that crosses to a different component (its cheapest outgoing edge, or COE). Break ties deterministically — here by edge index.

Merge. Add every selected COE to the MST and merge the two components it connects. Because multiple components may select the same edge, a Disjoint Set Union structure (Section 3.4) tracks which components have already merged.

After one round, every component that had an outgoing edge has merged with at least one neighbour, so the number of components at least halves per round. Starting from n components and halving each round, the algorithm terminates in at most $\lceil \log_2 n \rceil$ rounds. Each round scans all m edges, giving a total time of $O(m \log n)$ for a connected graph.

3.3 Graph Compression (Super-nodes and Super-edges)

After several Borůvka rounds the number of components has shrunk significantly — for example, from 1,000,000 vertices down to a few hundred “super-nodes”. It is wasteful to keep scanning all m original edges when most of them now connect vertices within the same super-node and can never be in the MST.

Graph compression solves this by replacing the current graph with a smaller compressed graph whose vertices are the surviving super-nodes. More precisely, if two super-nodes A and B are connected by several original edges, we keep only the minimum-weight one (by the cycle property, all others are dominated). Edges within a single super-node become self-loops and are discarded.

After compression the new graph has C vertices (one per remaining super-node) and at most $C - 1$ edges relevant to the MST. For sparse graphs, C shrinks rapidly (see Lemma 1), so subsequent Borůvka rounds operate on a much smaller edge set. This is the primary reason BMSBoruvka outperforms plain Borůvka on sparse graphs.

The algorithm maintains a mapping `origToComp[v]` that records, for each original vertex v , the ID of the super-node it currently belongs to. This allows new edges to be pushed into the compressed graph without re-reading the original adjacency list.

3.4 Disjoint Set Union (DSU)

A Disjoint Set Union (DSU), also called Union-Find, is a data structure that efficiently tracks which elements belong to the same set (component). It supports two operations:

find(x) — returns the representative (root) of the set containing x. Two elements are in the same component if and only if their representatives are equal.

unite(a, b) — merges the sets containing a and b, returning the new representative.

This implementation uses two standard optimisations. Union by rank [Tar75] always attaches the shorter tree under the taller one, keeping trees shallow. Path halving [TvL84] during find() rewires every other node on the path directly to its grandparent, compressing future traversals. Together these give an amortised cost of $O(\alpha(n))$ per operation, where α is the inverse Ackermann function — a value no larger than 4 for any input that fits in computer memory. For all practical purposes this is $O(1)$.

Crucially, find() is not a comparison in the graph-theory sense — it performs pointer traversals through the parent[] array. This distinction matters for the cost model (Section 3.6).

3.5 Selection vs. Sorting: nth_element

A classical result in algorithm theory is that selecting the k-th smallest element of an array of n items requires only $O(n)$ comparisons in the worst case, whereas sorting the entire array requires $\Omega(n \log n)$ comparisons. This gap of a factor $\log n$ is exactly what BMSBoruvka exploits.

The C++ standard library function `std::nth_element(first, nth, last, cmp)` rearranges the range [first, last) so that:

- The element at position nth is the element that would be there in sorted order.
- Every element before nth is no greater than the element at nth.
- Every element after nth is no less than the element at nth.
- Elements before nth and elements after nth are not sorted relative to each other.

The standard library guarantees worst-case $O(n)$ time for this operation using an algorithm called Introselect, which combines the average-case-fast Quickselect with a worst-case-optimal Median-of-Medians fallback. Crucially, the bound is unconditional — it holds regardless of the distribution or repetition of edge weights.

In BMSBoruvka, `nth_element` is called once per super-step on the unactivated suffix of the edge array. It moves the lightest k edges to the front (unordered) so they can be processed in that super-step, while deferring heavier edges to later super-steps. This preserves the weight-prefix property needed for correctness (Section 4.2) without paying the $O(m \log m)$ cost of a full global sort.

3.6 Cost Model

We analyse BMSBoruvka under the unit-cost word RAM model. In this model every elementary operation on a machine word — integer arithmetic, array indexing, pointer dereference, hash-table lookup — costs exactly 1 unit of time, regardless of its type. All integers, edge weights, and array indices fit in a single word.

Under this model:

- An edge-weight comparison costs 1 unit.
- A DSU find() call costs $O(\alpha(n))$ units (amortised over any sequence of operations), which we treat as $O(1)$ throughout the analysis since $\alpha(n) \leq 4$ in practice.
- A hash_map insertion or lookup costs $O(1)$ units amortised.
- Allocating or writing a block of k memory words costs $O(k)$ units.

This model is more general than the pure comparison model used in classical MST lower bounds (which counts only edge-weight comparisons). The distinction matters here because Phase B spends roughly four times as many units on DSU operations as on comparisons, and Phase C additionally incurs hash-table costs. The RAM model captures all of these.

The RAM model does not capture cache effects. In practice, find() accesses the parent[] array in a graph-dependent, near-random order; for large n (e.g., $n = 10^6$, parent[] occupying 4 MB) this causes L2/L3 cache pressure that is invisible to the RAM model. This explains why the empirical throughput metric $\hat{t}$ (operations per millisecond) trends upward from $n = 10,000$ to $n = 100,000$ before stabilising: at small n the DSU tree is shallower and the working set fits in cache; at large n both effects plateau together. The asymptotic bounds derived under the RAM model are confirmed by the empirical stabilisation of $\hat{t}$ for $n \geq 100,000$.

3.7 Summary of Notation

The following symbols are used consistently throughout Sections 4 and 5 as shown in Table 1.

Table 1. Summary of Notation.

Symbol	Meaning
$G = (V, E, w)$	Input graph: vertex set V , edge set E , weight function $w : E \rightarrow \mathbb{Z}$
n	Number of vertices, $n = V $
m	Number of edges, $m = E $
MST / MSF	Minimum spanning tree (connected graph) or minimum spanning forest (disconnected)
t	Number of Borůvka rounds per super-step: $t = \left\lceil \log_2^{\frac{2}{3}}(n) \right\rceil$
P	Number of super-steps: $P = \lceil \log_2(n)/t \rceil = \Theta\left(\log^{1/3} n\right)$
k	Edges activated per super-step: $k = \lfloor m/P \rfloor$

Symbol	Meaning
C_i	Number of super-nodes (components) at the start of super-step i ; $C_0 = n$
M_i	Size of <code>curEdges</code> entering Phase B of super-step i
m_i	Number of distinct inter-component edges after <code>compress()</code> in super-step i
<code>origToComp[v]</code>	Maps original vertex v to its current super-node ID
<code>curEdges</code>	Working edge list in the compressed graph, updated each super-step
$\alpha(n)$	Inverse Ackermann function; $\alpha(n) \leq 4$ for all practical n
$O(\cdot)$	Standard asymptotic upper bound (worst-case unless stated otherwise)

4 Results

4.1 The Algorithm

The BMSBoruvka algorithm operates on an undirected weighted graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges. Two integer parameters govern the structure of the computation:

$$t = \left\lceil \log_2^{\frac{2}{3}}(n) \right\rceil \geq 1 \text{ (Borůvka rounds per super-step)}$$

$$P = \lceil \log_2(n)/t \rceil = \Theta(\log^{1/3} n) \text{ (number of super-steps)}$$

The algorithm proceeds in P super-steps. Each super-step consists of three phases:

1. **Phase A (Activation).** A call to `nth_element` partitions the as-yet unactivated suffix of `edges[]` so that the $k = \lfloor m/P \rfloor$ lightest edges move to the front (unordered among themselves) and every remaining edge is no lighter than any activated edge. These are pushed into `curEdges` via the `origToComp[]` mapping. Because the activated prefix is always a weight-prefix of E , the cut property is preserved.
2. **Phase B (Borůvka Rounds).** Execute t standard Borůvka rounds on `curEdges`. In each round every component selects its minimum-weight outgoing edge, all selected edges are added to the MST, and edges whose endpoints have been merged are filtered out.
3. **Phase C (Compression).** `compress()` rennumbers the surviving super-nodes $0, \dots, C_{i+1}-1$, rebuilds `curEdges` with the new IDs (discarding self-loops), and updates `origToComp[0..n]`. The DSU is then reinitialised in the compressed space.

The outer loop terminates when all $n-1$ MST edges have been collected, when `uf.comps` drops to 1, or after at most P super-steps.

4.2 Correctness Analysis

Theorem 1 (MST / MSF Correctness). *BMSBoruvka returns a minimum spanning tree of G if G is connected, and a minimum spanning forest otherwise.*

Proof. We verify the three invariants that together guarantee correctness. □

Cut property preservation. `nth_element` guarantees that `edges[curActivated..activateEnd)` contains the globally lightest k unactivated edges: every edge in the window has weight $\leq$ every edge outside it. Because `curActivated` advances monotonically across super-steps, the cumulative activated set is always a weight-prefix of E . The cut property requires only that the minimum-weight crossing edge of each cut be present in `curEdges` at the moment that cut is resolved; since the activated set is a weight-prefix, this condition is always satisfied.

Phase B safety. Each Borůvka round selects, for every active component, its cheapest outgoing edge. By the cut property of matroids, each such edge is safe (i.e., belongs to some MST). No edge is selected twice because self-loops are filtered before each round.

Compression correctness. Every edge in the compressed graph corresponds to a unique original edge via `orig_idx`. Self-loops (edges whose endpoints lie in the same component) are discarded, which is sound because a self-loop cannot belong to any spanning tree. The `origToComp[]` mapping is updated consistently so that future Phase A activations route edges to the correct super-nodes.

For disconnected graphs the outer loop terminates once no component has an outgoing edge in `curEdges`; the result is a minimum spanning forest with $n - k$ edges, where k is the number of connected components. Correctness was independently verified by 3,000 random stress tests against Kruskal's algorithm.

4.3 Time Complexity Analysis

Lemma 1 (Component Shrinkage). *After super-step i , the component count satisfies $C_{i+1} \leq C_i / 2^t$. In particular, $C_P = 1$ for a connected graph, and $C_i \leq \frac{n}{2^{it}}$.*

Proof. Each of the t Borůvka rounds in Phase B selects, for every component, its minimum-weight outgoing edge; by the cut property every selected edge is safe. After one such round every component that possesses an outgoing edge merges with at least one neighbour, so the component count at least halves. Repeating t times gives $C_{i+1} \leq \frac{C_i}{2^t}$. Since $t \cdot P = \Theta(\log n)$, after P super-steps we have $C_P \leq \frac{n}{2^{tP}} \rightarrow 1$. □

Lemma 2 (`nth_element` Total Cost). *The aggregate cost of all `nth_element` calls is $O(mP) = O\left(m \log^{1/3} n\right)$.*

Proof. Super-step i operates on the suffix of size $m - i \cdot k$; `introsselect` guarantees worst-case $O(\text{size})$ time independent of weight distribution. Summing the arithmetic progression over steps 0 through $P-2$ (the final step activates all remaining edges without a partition call): $\sum_{i=0}^{P-2} (m - i \cdot k) \leq \frac{m(P+1)}{2} = O(mP) = O\left(m \log^{1/3} n\right)$. No degenerate case exists; the bound is unconditional. □

Lemma 3 (Phase A Total Cost). *The aggregate cost of all Phase A activation loops is $O(m)$.*

Proof. Each original edge is pushed into `curEdges` at most once (`curActivated` advances monotonically) and each `origToComp` lookup is $O(1)$. The total work across all super-steps is therefore $O(m)$. $\square$

Lemma 4 (Phase B, General Graphs). *The total cost of all Phase B Borůvka rounds is $O(m \log n)$.*

Proof. Let $M_i^{(r)}$ denote the size of `curEdges` at the start of round r within super-step i . Each round scans `curEdges` in $O(M_i^{(r)})$ time and performs DSU operations in $O(C_i)$. Since each edge survives at most t rounds before its endpoints merge, $\sum_r M_i^{(r)} \leq t \cdot M_i$. Applying the crude bound $M_i \leq m$ and summing over all super-steps: $T_{\text{Phase B}} \leq O(t \cdot P \cdot m) = O(\log^{2/3}(n) \cdot \log^{1/3}(n) \cdot m) = O(m \log n)$. $\square$

Lemma 5 (Phase B, Sparse Graphs). *For sparse graphs with $m = O(n)$, the total Phase B cost is $O(n \log^{2/3} n)$.*

Proof. Under the sparse assumption $m'_i = O(C_i)$ (the compressed graph retains $O(1)$ average super-node degree), we have $M_i \leq m'_i + k = O(C_i + k)$. Summing: $\sum_i M_i \leq \sum_i C_i + m$. By Lemma 1, $\sum_i C_i \leq \frac{n}{1-2^{-t}} \leq 2n$. With $m = O(n)$, $\sum_i M_i = O(n)$, so the total Phase B cost is $O(t \cdot n) = O(n \log^{2/3} n)$. $\square$

Lemma 6 (Phase C Total Cost). *The total cost of all `compress()` calls is $O(\sum_i M_i + P \cdot n)$.*

Proof. Each `compress()` call performs: (i) an $O(M_i)$ scan to filter self-loops; (ii) $O(C_i)$ work to enumerate roots and build the renaming map; (iii) $O(m'_i)$ edge renaming; (iv) an $O(n)$ `origToComp` update; (v) an $O(C_i + C_{i+1})$ array reset. Summing over P super-steps yields $O(\sum_i M_i + P \cdot n)$. For general graphs this is $O(m \log^{1/3} n)$; for sparse graphs, $O(n \log^{1/3} n)$. $\square$

Theorem 2 (Sparse Graphs, $m = O(n)$). *BMSBoruvka computes an MST in time $T = O(n \log^{2/3} n)$.*

Theorem 3 (General Graphs). *BMSBoruvka computes an MST in time $T = O(m \log n)$.*

Proof of Theorem 2. Collecting costs for $m = O(n)$: $\square$

Table 2. Cost breakdown of BMSBoruvka on sparse graphs.

Component	Cost (sparse, $m = O(n)$)	Dominant?
<code>nth_element</code> (Lemma 2)	$O(n \log^{1/3} n)$	No
Phase A activation (Lemma 3)	$O(n)$	No
Phase B Borůvka (Lemma 5)	$O(n \log^{2/3} n)$	Yes

Component	Cost (sparse, $m = O(n)$)	Dominant?
Phase C compress (Lemma 6)	$O\left(n \log^{1/3} n\right)$	No
Total	$O\left(n \log^{2/3} n\right)$	

Proof. Phase B dominates since $\log^{1/3}(n) \leq \log^{2/3}(n)$ for all $n \geq 2$, so every other component lies in $O\left(n \log^{1/3} n\right) \subseteq O\left(n \log^{2/3} n\right)$. $\square$

Proof of Theorem 3. For general graphs, apply the crude bound $M_i \leq m$ in Lemma 4: $\square$

Table 3. Cost breakdown of BMSBoruvka on general graphs.

Component	Cost (general)	Dominant?
nth_element (Lemma 2)	$O\left(m \log^{1/3} n\right)$	No
Phase A activation (Lemma 3)	$O(m)$	No
Phase B Borůvka (Lemma 4)	$O(m \log n)$	Yes
Phase C compress (Lemma 6)	$O\left(m \log^{1/3} n\right)$	No
Total	$O(m \log n)$	

Proof. Phase B dominates: $O(t \cdot P \cdot m) = O(\log(n) \cdot m)$. $\square$

Remark 1 (Sparse-graph condition). The bound $O\left(n \log^{2/3} n\right)$ requires $m = O(n)$ (bounded average degree). This is needed to ensure $m'_i = O(C_i)$ after each compression, which in turn gives $\sum_i M_i = O(n)$. For denser graphs $m = O(n f(n))$, $f(n) \rightarrow \infty$, Phase B costs $O\left(n \log^{2/3}(n) \cdot f(n)\right)$, which still improves on the plain Borůvka bound $O(m \log n) = O(n f(n) \log n)$ by a factor of $\log^{1/3} n$.

5 Benchmark

5.1 Set up

Binaries were compiled with `g++ -O2 -std=c++17`, and the parallel variant additionally with `-fopenmp`. Three algorithms were evaluated:

1. **boruvka**. Standard sequential Borůvka, implemented with the same DSU and edge-comparison routines as BMSBoruvka. Serves as the baseline.
2. **par_Tk**. OpenMP-parallelised Borůvka with k threads ($k \in \{2, 4\}$). Phase 1 parallelises the COE scan with thread-local best-edge arrays; Phase 2 reduces them sequentially; Phase 3 unions sequentially.
3. **bms**. BMSBoruvka (v6, nth_element preprocessing), the algorithm analysed in Section 4.

Test graphs were generated with a custom random graph generator using seed 42. Every graph was guaranteed to be connected (a random spanning path was embedded first). Two density classes were evaluated:

1. **Sparse ($m \approx 5n$).** $n \in \{10000, 100000, 500000, 1000000\}$, $m = 5n$.
2. **Dense ($m \approx 50n$).** $n \in \{50000, 100000\}$, $m = 50n$.

Timing used `time.perf_counter()` in Python with a one warm-up run discarded before each (algorithm, test) combination. The number of timed trials was determined adaptively: a probe run estimated the single-run latency, and trials were set to `clamp([24000 ms ÷ budget], 3, 60)`, targeting roughly two minutes of total measurement time per test case across all algorithms. Program output was redirected to `/dev/null` to exclude I/O from timings.

5.2 Set up

Wall-clock time. Mean, minimum, maximum, and standard deviation (σ) of timed runs, reported in milliseconds.

Shrinkage factor ξ . To quantify how quickly each algorithm reduces the graph, we define $\xi_i = \frac{C_i}{C_{i+1}}$, where C_i is the number of connected components before step i . For sequential Borůvka, the step is a single round; for BMSBoruvka, the step is a full super-step. We report ξ_μ , the geometric mean of all per-step ξ values over a complete run. A larger ξ_μ indicates faster component consolidation per step. The theoretical lower bound for one Borůvka round is $\xi \geq 2$ (components at least halve); for a BMSBoruvka super-step the theoretical value is $\xi = 2^t$.

Normalised throughput $\hat{t}$. To assess whether the empirical performance matches the theoretical complexity, we define $\hat{t} = \frac{\theta_0(n,m)}{T_{\text{mean}}}$, where T_{mean} is the mean wall-clock time in milliseconds and θ_0 is the algorithm's theoretical operation count:

$$\text{boruvka / par: } \theta_0 = m \cdot \log_2(n) \quad [O(m \log n)]$$

$$\text{bms: } \theta_0 = n \cdot \log_2^{\frac{2}{3}}(n) \quad \left[O\left(n \log^{\frac{2}{3}} n \right) \right]$$

If $\hat{t}$ converges to a constant as n grows, the theoretical complexity bound is tight in practice. A rising $\hat{t}$ indicates improving hardware utilisation; a falling $\hat{t}$ would indicate growing overhead relative to the theoretical model.

5.3 Shrinkage Factor ξ

Table 4 reports ξ_μ (averaged over three independent runs) for the sparse graph series.

Table 4. Measured shrinkage factor ξ_μ (geometric mean) for sparse graphs ($m \approx 5n$).

n	m	Algorithm	ξ_μ (measured)	ξ_{theory}	Steps / Rounds	t
10,000	50,000	bms	40.82	64	2	6

n	m	Algorithm	ξ_μ (measured)	ξ_{theory}	Steps / Rounds	t
10,000	50,000	boruvka	5.07	2	5	—
100,000	500,000	bms	45.64	128	2	7
100,000	500,000	boruvka	5.67	2	6	—
500,000	2,500,000	bms	44.19	256	2	8
500,000	2,500,000	boruvka	5.35	2	7	—
1,000,000	5,000,000	bms	42.84	256	2	8
1,000,000	5,000,000	boruvka	5.90	2	7	—

Several observations follow from Table 4. First, BMSBoruvka consistently terminates in only two super-steps for all sizes tested, compared to five to seven rounds for sequential Boruvka. This is a direct consequence of the large per-super-step shrinkage: with $t = 8$ and $\xi_\mu \approx 43$ for $n = 10^6$, two super-steps reduce 10^6 components to approximately $10^6/43^2 \approx 540$, and the residual is absorbed by the second step.

Second, the measured ξ_μ for BMSBoruvka (41–46) is substantially smaller than the theoretical value 2^t (64–256). This is expected: the theoretical value assumes every super-step runs all t Boruvka rounds to completion, but in practice the algorithm terminates early within a super-step as soon as no merges are possible. The measured ξ_μ therefore reflects the actual convergence rate rather than the worst-case guarantee.

Third, sequential Boruvka achieves $\xi_\mu \approx 5 - 6$ per round on these random sparse graphs, well above the theoretical lower bound of 2. This is characteristic of random graphs, where many components simultaneously find outgoing edges to previously isolated vertices, producing merges that exceed the minimum halving guarantee.

5.4 Wall-Clock Time and Speedup

Tables 5 and 6 report the mean runtime, standard deviation, and speedup ratios for sparse and dense graph instances respectively.

Table 5. Runtime results for sparse graphs ($m \approx 5n$). ‘vs boruvka’ = boruvka mean / algo mean; ‘vs bms’ = bms mean / algo mean. Ratios > 1 indicate the algorithm is faster than the reference.

n	m	trials	Algorithm	Mean (ms)	σ	vs boruvka	vs bms
10,000	50,000	25	boruvka	80.8	11.1	—	0.41×
10,000	50,000	25	bms	33.1	14.3	2.44×	—
10,000	50,000	25	par_T2	92.0	10.1	0.88×	0.36×
10,000	50,000	25	par_T4	91.3	8.6	0.88×	0.36×
100,000	500,000	6	boruvka	722.9	26.7	—	0.37×
100,000	500,000	6	bms	268.3	13.1	2.69×	—
100,000	500,000	6	par_T2	839.6	34.6	0.86×	0.32×
100,000	500,000	6	par_T4	813.1	21.9	0.89×	0.33×
500,000	2,500,000	3	boruvka	3870.6	78.7	—	0.36×
500,000	2,500,000	3	bms	1404.3	2.0	2.76×	—

n	m	trials	Algorithm	Mean (ms)	σ	vs boruvka	vs bms
500,000	2,500,000	3	par_T2	4109.9	16.2	0.94×	0.34×
500,000	2,500,000	3	par_T4	4150.9	56.8	0.93×	0.34×
1,000,000	5,000,000	3	boruvka	7950.6	61.5	—	0.37×
1,000,000	5,000,000	3	bms	2921.4	62.6	2.72×	—
1,000,000	5,000,000	3	par_T2	8556.6	47.6	0.93×	0.34×

Table 6. Runtime results for dense graphs ($m \approx 50n$).

n	m	trials	Algorithm	Mean (ms)	σ	vs boruvka	vs bms
50,000	2,500,000	3	boruvka	3137.6	41.8	—	0.37×
50,000	2,500,000	3	bms	1160.7	11.1	2.70×	—
50,000	2,500,000	3	par_T2	3385.9	10.5	0.93×	0.34×
50,000	2,500,000	3	par_T4	3423.9	50.7	0.92×	0.34×
100,000	5,000,000	3	boruvka	6430.7	22.7	—	0.39×
100,000	5,000,000	3	bms	2495.4	33.3	2.58×	—
100,000	5,000,000	3	par_T2	6749.1	23.7	0.95×	0.37×

BMSBoruvka vs boruvka (sparse). BMSBoruvka achieves a speedup of 2.44× to 2.76× over sequential Borůvka across all sparse instances. The speedup is consistent and grows slightly with n , rising from 2.44 at $n = 10000$ to 2.76 at $n = 500000$. This trend reflects the increasing advantage of graph compression as the number of Borůvka rounds required grows with n : while boruvka must scan all m edges in each of its five to seven rounds, BMSBoruvka compresses the graph after each super-step and subsequent rounds operate on a much smaller edge set.

Parallel Borůvka vs sequential (sparse). The parallel variants (par_T2, par_T4) are consistently slower than sequential Borůvka on this two-core machine. Both par_T2 and par_T4 incur approximately 12–15% overhead relative to sequential at $n = 100000$, shrinking to 6–7% at $n = 1000000$. The overhead arises from three sources: (i) allocation and initialisation of $T \times n$ thread-local best-edge arrays per round, costing $O(Tn)$ per super-step; (ii) the sequential Phase 2 reduction over all $T \times n$ entries; and (iii) OpenMP thread-launch latency. With only two physical cores, the parallel workload in Phase 1 is insufficient to amortise these costs. On machines with 16 or more cores, where Phase 1 can distribute m edge operations over many threads, the parallel variant is expected to show positive speedup.

Dense graphs. On dense instances ($m \approx 50n$), BMSBoruvka is still 2.58×2.70× faster than sequential Borůvka in absolute wall-clock time. However, as discussed in Section 5.5, this speedup no longer corresponds to a theoretical complexity advantage: on dense graphs Phase B degrades to $O(m \log n)$ regardless of algorithm, so both boruvka and BMSBoruvka are asymptotically equivalent. The practical speedup observed here stems from BMSBoruvka’s fewer total rounds (due to larger per-super-step shrinkage) rather than from reduced edge-scan work per round.

5.5 Throughput $\hat{t}$ and Complexity Confirmation

Table 7 reports $\hat{t}$ and ξ_μ for the sparse series, tracking both metrics as n grows.

Table 7. Shrinkage factor ξ_μ and normalised throughput $\hat{t}$ for sparse graphs.

$$\hat{t}_{bms} = \frac{n \log_2^{\frac{2}{3}} n}{T_{\text{mean}}}; \hat{t}_{boruvka} = \frac{m \log_2 n}{T_{\text{mean}}}.$$

n	$\xi_\mu(\text{bms})$	$\xi_{theory}(2^t)$	$\xi_\mu(\text{boruvka})$	$\hat{t}_{bms}(\text{ops/ms})$	$\hat{t}_{boruvka}(\text{ops/ms})$
10,000	40.82	64	5.07	1,695	8,223
100,000	45.64	128	5.67	2,426	11,488
500,000	44.19	256	5.35	2,529	12,228
1,000,000	42.84	256	5.90	2,516	12,535

$\hat{t}_{boruvka}$ rises from 8,223 at $n = 10000$ to 12,535 at $n = 1000000$, stabilising in the range 11,000–13,000 for $n \geq 100000$. This convergence confirms that the $O(m \log n)$ bound is tight: the algorithm processes a consistent number of theoretical operations per millisecond at scale, with no hidden super-linear growth term.

$\hat{t}_{bms}$ rises from 1,695 to 2,516 and stabilises in the range 2,400–2,530 for $n \geq 100000$. This confirms that the $O\left(n \log^{\frac{2}{3}} n\right)$ bound is tight for BMSBoruvka on sparse graphs. The lower absolute value of $\hat{t}_{bms}$ compared to $\hat{t}_{boruvka}$ is not a contradiction: the two normalisations use different theoretical operation counts ($n \log^{\frac{2}{3}} n$ versus $m \log n$) that are not directly comparable in magnitude.

The upward trend in both $\hat{t}$ values at small n is explained by cache behaviour. At $n = 10000$, the DSU parent[] array occupies only 40 KIB and fits comfortably in L1/L2 cache; find() calls resolve with minimal memory latency. As n grows beyond the cache boundary, find() accesses become cache misses, increasing the real cost per theoretical operation. By $n = 100000$ both effects have plateaued and $\hat{t}$ stabilises, confirming that the RAM model captures the asymptotic regime correctly even though it does not model cache effects.

Dense graph behaviour. For dense graphs, $\hat{t}_{bms}$ drops to approximately 260–270, far below the sparse-graph values. This is an artefact of the normalisation: $\hat{t}_{bms}$ uses $n \log^{\frac{2}{3}} n$ as its theoretical operation count, which is derived under the sparse assumption $m = O(n)$. On dense graphs Phase B costs $O(m \log n)$ (Theorem 3), not $O\left(n \log^{\frac{2}{3}} n\right)$. Normalising by the smaller sparse formula therefore makes $\hat{t}_{bms}$ appear artificially low. This is precisely the intended behaviour: $\hat{t}_{bms} \ll \hat{t}_{boruvka}$ on dense graphs serves as an empirical signal that the theoretical advantage of BMS-Boruvka does not apply, consistent with the dense-graph remark following Theorem 3.

5.6 Summary

Table 8. Summary of experimental findings.

Metric	Sparse graphs ($m \approx 5n$)	Dense graphs ($m \approx 50n$)
BMSBoruvka speedup vs boruvka	$2.44\times - 2.76\times$	$2.58\times - 2.70\times$
Parallel Borůvka vs boruvka (2-core)	$0.86\times - 0.94\times$	$0.92\times - 0.95\times$
$\hat{t}_{bms}$ converges?	Yes, ~ 2400 - 2530 ($n \geq 1e5$)	No (wrong normalisation basis)
$\hat{t}_{boruvka}$ converges?	Yes, ~ 11000 - 13000 ($n \geq 1e5$)	Yes, ~ 12300 - 13000
$O(m \log n)$ bound tight?	Confirmed ($\hat{t}_{boruvka}$ stable)	Confirmed ($\hat{t}_{boruvka}$ stable)
$O\left(n \log^{\frac{2}{3}} n\right)$ bound tight?	Confirmed ($\hat{t}_{bms}$ stable)	N/A (dense graph, $m \neq O(n)$)
BMSBoruvka super-steps boruvka rounds	2 (all sizes) 5-7	0-2 6

The experiments confirm all major theoretical predictions. BMSBoruvka achieves a consistent $2.5\times$ - $2.8\times$ wall-clock speedup over sequential Borůvka on sparse graphs, and the normalised throughput $\hat{t}$ stabilises from $n = 100000$ onward for both algorithms, confirming that $O\left(n \log^{\frac{2}{3}} n\right)$ and $O(m \log n)$ are empirically tight bounds. The parallel variant does not improve on sequential performance at two cores due to per-round allocation overhead and the serial reduction phase, but the underlying Phase 1 scan is embarrassingly parallel and is expected to scale on machines with more cores. On dense graphs, BMSBoruvka retains a practical speedup but loses its theoretical complexity advantage, as predicted by the sparse-graph condition $m = O(n)$ established in Lemma 5.

6 Discussion

6.1 Comparison with Classical MST Algorithms

The minimum spanning tree problem has been studied for nearly a century, and the evolution of its algorithms reflects a steady push towards lower asymptotic complexity. Table 9 places BMSBoruvka in the context of the major results in this history, across three dimensions: worst-case time complexity, cost model, and practical implementability.

Table 9. Selected MST algorithms in chronological order. α = inverse Ackermann function; $T^*(m,n)$ = decision-tree complexity of MST (exact value unknown).

Algorithm	Time complexity	Model	Randomised?	Practical?
Borůvka [Bor26]	$O(m \log n)$	Comparison	No	Yes
Kruskal [Kru56]	$O(m \log n)$	Comparison	No	Yes
Prim [Pri57] + Fibonacci heap [FT87]	$O(m + n \log n)$	Comparison	No	Yes

Algorithm	Time complexity	Model	Randomised?	Practical?
Karger, Klein & Tarjan [KKT95]	$O(m)$ expected	Comparison	Yes	Limited
Chazelle [Cha00]	$O(m \alpha(m, n))$	Comparison	No	No
Pettie & Ramachandran [PR02]	$O(T^*(m, n))$	Comparison	No	No
BMSBoruvka (this work)	$O\left(n \log^{\frac{2}{3}} n\right)$ [sparse]	RAM	No	Yes
BMSBoruvka (this work)	$O(m \log n)$ [general]	RAM	No	Yes

Classical algorithms (**Borůvka** [Bor26], **Kruskal** [Kru56], **Prim** [Pri57]). The three textbook algorithms all run in $O(m \log n)$ or $O(m + n \log n)$ time under the comparison model. Borůvka’s algorithm [Bor26], the structural foundation of BMSBoruvka, iterates rounds in which every component simultaneously selects its cheapest outgoing edge; after $\lceil \log_2 n \rceil$ rounds a single component remains. Kruskal’s algorithm [Kru56] sorts all edges once and greedily adds them in order, with cost dominated by the $O(m \log m)$ sort. Prim’s algorithm [Pri57], when implemented with a Fibonacci heap [FT87], achieves the best classical bound of $O(m + n \log n)$, which is tighter on dense graphs but equivalent to Borůvka on sparse ones where $m = O(n)$. All three are straightforward to implement and have been standard in pedagogy and practice for decades. BMSBoruvka improves on all three for sparse graphs, achieving $O\left(n \log^{\frac{2}{3}} n\right)$ versus the best classical bound of $O(n \log n)$ when $m = O(n)$, an improvement by a factor of $\log^{1/3} n$.

Karger, Klein & Tarjan (1995) [KKT95]. Karger, Klein, and Tarjan gave the first linear-time MST algorithm, achieving $O(m)$ expected time under the comparison model by combining random edge sampling with a linear-time MST verification subroutine. The algorithm is optimal in expectation: no comparison-based algorithm can do better than $\Omega(m)$ in expectation. However, its reliance on randomness is a meaningful drawback. The algorithm consumes $O(m)$ random bits and uses MST verification as a black box, making it substantially more complex to implement correctly than the classical algorithms or BMSBoruvka. In practice, the large constant factors and the overhead of the verification step mean that the algorithm is rarely competitive on realistic inputs despite its superior asymptotic bound. BMSBoruvka, while asymptotically slower on general graphs, is deterministic, requires no random bits, and is faster in practice on the sparse graph inputs most common in engineering applications.

Chazelle (2000) [Cha00]. Chazelle’s algorithm is the best known deterministic MST algorithm in the comparison model, running in $O(m \alpha(m, n))$ time, where α is the inverse Ackermann function. Since $\alpha(m, n) \leq 4$ for any input that fits in computer memory, this is effectively $O(m)$ in practice. The algorithm is built around the soft heap, an approximate priority queue that deliberately corrupts a small fraction of its keys in exchange for $O(1)$ amortised operation time. Despite its elegance, Chazelle’s algorithm has never been implemented in practice: the soft heap introduces enormous constant factors, the algorithm is designed for a pointer machine rather than a

standard RAM, and the correctness argument is extremely delicate. BMSBoruvka is asymptotically inferior to Chazelle on sparse graphs ($O\left(n \log^{\frac{2}{3}} n\right)$ vs $O(m \alpha(m, n))$) but offers something Chazelle’s algorithm does not: a working implementation. Using only a DSU and `std::nth_element`, BMSBoruvka achieves $O\left(n \log^{\frac{2}{3}} n\right)$ on sparse graphs with a codebase under 400 lines of standard C++, verified correct on 3,000 random test cases.

Pettie & Ramachandran (2002) [PR02]. Pettie and Ramachandran proved that the algorithmic complexity of MST equals its decision-tree complexity, presenting a deterministic algorithm running in $O(T^*(m, n))$ time, where $T^*(m, n)$ is the minimum number of edge-weight comparisons needed to determine the MST on any graph with n vertices and m edges. This algorithm is provably optimal in the comparison model. The current best bounds on T^* are $\Omega(m) \leq T^*(m, n) \leq O(m \alpha(m, n))$, so the exact complexity of MST remains unknown. The algorithm is purely theoretical: it operates by precomputing optimal decision trees for all graphs up to a fixed size r , and the constant factor is exponential in r . No practical implementation exists or is expected. BMSBoruvka makes no claim to theoretical optimality, but its $O\left(n \log^{\frac{2}{3}} n\right)$ sparse bound is derived rigorously, and the algorithm runs correctly on inputs of up to $n = 10^6$ vertices in under 3 seconds on commodity hardware.

6.2 Positioning of BMSBoruvka

The comparison above reveals a clear division in the MST algorithm landscape. On one side are the theoretically superior algorithms—Karger-Klein-Tarjan [KKT95] ($O(m)$ expected) and Chazelle [Cha00] ($O(m \alpha(m, n))$)—that achieve lower asymptotic bounds but are either randomised, impractical to implement, or both. On the other side are the classical practical algorithms—Borůvka [Bor26], Kruskal [Kru56], and Prim [Pri57]—that are widely deployed but plateau at $O(m \log n)$.

BMSBoruvka occupies a distinct middle position. For sparse graphs ($m = O(n)$), it achieves $O\left(n \log^{\frac{2}{3}} n\right)$, which is asymptotically better than every classical algorithm

and better than Prim on sparse graphs by a factor of $\log^{\frac{1}{3}} n$. It is deterministic, requires no random bits, and its implementation uses only standard data structures available in any modern programming environment. Empirically, it achieves $2.44\times$ to $2.76\times$ speedup over sequential Borůvka on sparse inputs (Section 5), and the normalised throughput $\hat{t}$ stabilises above $n = 100,000$, confirming that the $O\left(n \log^{\frac{2}{3}} n\right)$ bound is empirically tight.

On general graphs, BMSBoruvka degrades to $O(m \log n)$, matching Borůvka [Bor26]. This is expected: the $\log^{\frac{2}{3}}$ advantage arises specifically from graph compression on sparse inputs, where the compressed super-graph shrinks rapidly (Lemma 5). On dense graphs, Phase B cannot reduce the edge set significantly through compression, and the algorithm reverts to the same asymptotic regime as its classical predecessors. This limitation is consistent with the behaviour of all known sub- $O(m \log n)$ MST algorithms: they either require randomness ([KKT95]) or are impractical ([Cha00], [PR02]) to transcend the $O(m \log n)$ bound on dense inputs in

a deployable form.

References

- [Bor26]** Borůvka, O. (1926). O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, 3, 37–58.
- [Cha00]** Chazelle, B. (2000). A minimum spanning tree algorithm with inverse-Ackermann type complexity. *Journal of the ACM*, 47(6), 1028–1047. <https://doi.org/10.1145/355541.355562>
- [DMMS+25]** Duan, R., Mao, J., Mao, X., Shu, X., & Yin, L. (2025). *Breaking the sorting barrier for directed single-source shortest paths*. arXiv. <https://doi.org/10.48550/arXiv.2504.17033>
- [FT87]** Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3), 596–615. <https://doi.org/10.1145/28869.28874>
- [Hil24]** Hillyard, R. C. (2024). *Data structures, algorithms, and invariants: A practical guide*. Cognella Academic Publishing. ISBN 978-1-7935-8884-5.
- [KKT95]** Karger, D. R., Klein, P. N., & Tarjan, R. E. (1995). A randomized linear-time algorithm to find minimum spanning trees. *Journal of the ACM*, 42(2), 321–328. <https://doi.org/10.1145/201019.201022>
- [Kru56]** Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1), 48–50. <https://doi.org/10.1090/S0002-9939-1956-0078686-7>
- [PR02]** Pettie, S., & Ramachandran, V. (2002). An optimal minimum spanning tree algorithm. *Journal of the ACM*, 49(1), 16–34. <https://doi.org/10.1145/505241.505243>
- [Pri57]** Prim, R. C. (1957). Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6), 1389–1401. <https://doi.org/10.1002/j.1538-7305.1957.tb01515.x>
- [Tar75]** Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2), 215–225. <https://doi.org/10.1145/321879.321884>
- [TvL84]** Tarjan, R. E., & van Leeuwen, J. (1984). Worst-case analysis of set union algorithms. *Journal of the ACM*, 31(2), 245–281. <https://doi.org/10.1145/62.2160>